ASSIGNMENT 3 – TASK 6

Deadlock and Inter-Thread Communication

Programming in Java | CO3 | 7 Marks | K4 – Analyse

1. Problem interpretation

The task models hostel wallet transfers that lock two accounts and a mess counter where one producer adds meal tokens while consumers remove them. The counter has a maximum capacity of five.

2. Part A – Deadlock

The unsafe version allows one thread to lock wallet A and request B while another locks B and requests A. Both can wait forever. The four Coffman conditions are mutual exclusion, hold and wait, no preemption and circular wait.

Prevention: acquire both wallet locks in ascending account-ID order. Every transfer locks the lower ID first and the higher ID second. This breaks circular wait and preserves the total balance.

3. Part B – Producer–consumer counter

The buffer is a circular array of length five. count tracks occupancy; in is the insertion index and out is the removal index. put() waits while full, inserts a token, updates count and calls notifyAll(). take() waits while empty, removes a token, updates count, calls notifyAll() and returns the token.

Completed TODO 1 – put()

```
while (count == buffer.length) wait();\nbuffer[in] = token;\nin = (in + 1) % buffer.length;\ncount++;\nnotifyAll();
```

Completed TODO 2 – take()

```
while (count == 0) wait();\nint token = buffer[out];\nout = (out + 1) % buffer.length;\ncount--;\nnotifyAll();\nreturn t
```

4. wait(), while and monitor ownership

wait() must be called while owning the object's monitor. It releases that monitor while waiting and reacquires it before returning. The condition belongs in a while loop because wake-ups can be spurious or another thread can change the state before the awakened thread reacquires the monitor. if does not recheck the condition.

5. notify() versus notifyAll()

notify() wakes one arbitrary waiter. With several producers and consumers, it may wake a thread whose condition is still false. notifyAll() wakes all waiters, each of which competes to reacquire the monitor and rechecks its condition. It is the safer general choice for this design.

6. sleep() versus wait()

sleep() pauses the current thread and does not release monitors it owns. wait() releases the object's monitor while waiting and reacquires it before returning.

7. Alternative concepts and complexity

ArrayBlockingQueue(5) provides a bounded thread-safe queue. put() blocks when full and take() blocks when empty. Typical operations are O(1), with O(C) storage for capacity C. It is less error-prone than manually coordinating wait/notify.

A ReentrantLock alternative can use tryLock(timeout) for both accounts. If either lock cannot be acquired, any acquired lock must be released in finally and the transfer retried or reported as failed. Transfer work is O(1), excluding contention/wait time; lock storage is O(A) for A accounts.

8. Test cases and expected observations

Test	Expected observation
TokenCounter	20 tokens produced/consumed; final sum = 210.

Buffer bounds	Every recorded size remains between 0 and 5.
Unsafe transfer	Opposite-order locking can deadlock.
Safe transfer	Threads complete; total balance is preserved.
ArrayBlockingQueue	Capacity 5; final sum = 210.

The attached output files are sample executions. Re-run the programs on the submission computer and include screenshots and measurements from that environment. Deadlock observation can vary with scheduling.

9. Joint conclusion

Both problems coordinate access to shared state. Consistent lock ordering prevents circular wait in wallet transfers. The producer–consumer design uses synchronized state checks, wait loops and notification so operations proceed only when buffer capacity permits.